KLAi

Backpropagation by hand: the chain rule, run backwards through a tiny network

What happens in every training step of a language model, and why does training cost far more computing than answering a question?

Mathematics for AI and Machine Learning | Session 11 of 18 | Calculus and optimisation

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Compute the forward pass and the loss of a tiny network, with every number written out



Compute every gradient with the chain rule in the backward pass, apply the update, and see the loss fall



Write the same computation with the matrices W, b and delta, and check the gradients with finite differences in code

Your two hours: one network, forward then backward

0:00	10	Recall and a real question from AI
0:10	35	Concept: a picture, then a hand example
0:45	10	Break
0:55	45	Lab: Python or spreadsheet, predict, run, change
1:40	15	Where it appears in AI, plus an exit check
1:55	5	Save the work

Pictures and hand calculations first. Then the lab: numpy, or cells in the spreadsheet.

Three questions from last time

1. With eta = 0.1, old w = 2 and a slope of 5, what is the new w?

2. The loss grows at every step of gradient descent. What is the most likely cause?

3. What does Adam add to plain gradient descent?



Answer aloud first. Then we check together.

Answers to the three questions from Session 10

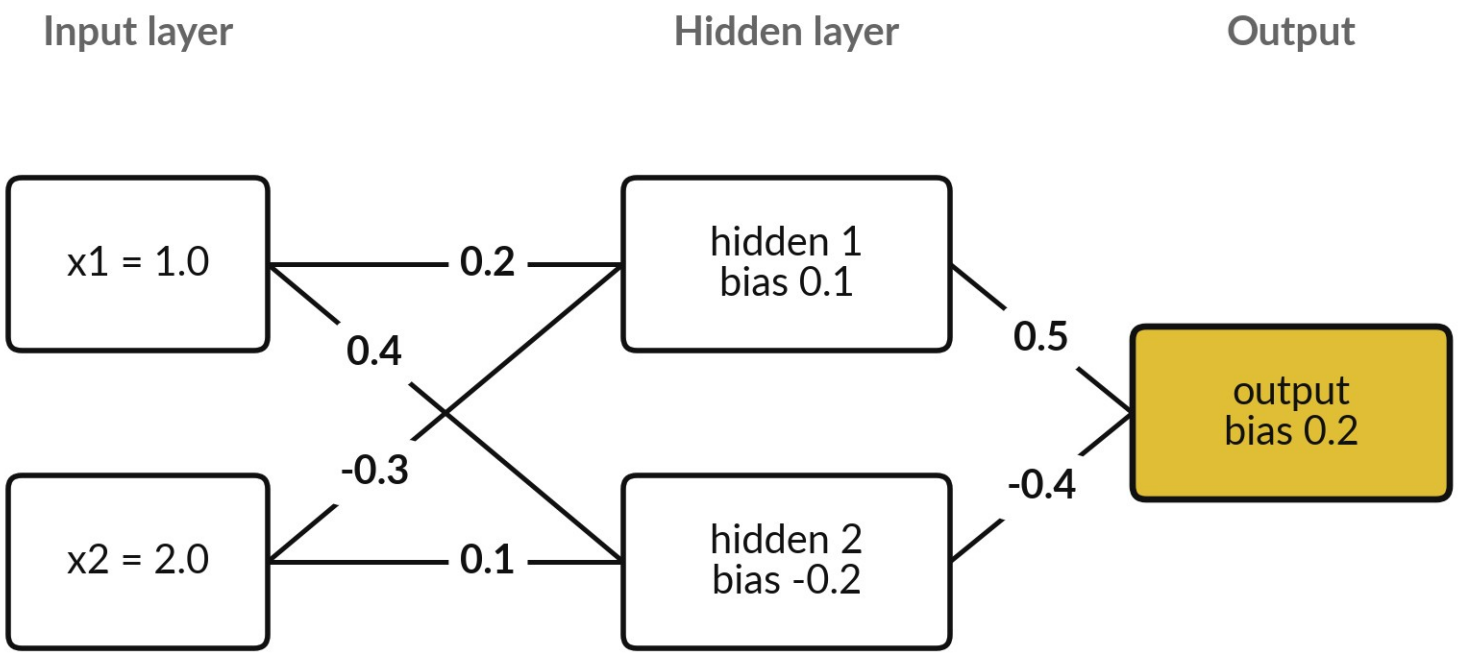
Question	Short answer
With eta = 0.1, old w = 2 and a slope of 5, what is the new w?	New w = 2 - 0.1 × 5 = 1.5.
The loss grows at every step of gradient descent. What is the most likely cause?	The learning rate is too large, so each step overshoots the bottom. Divide it by 3 or 10.
What does Adam add to plain gradient descent?	A running average of past gradients, and a separate step size for each parameter based on the typical size of its gradient.

Check your answers. If one surprised you, look at Session 10 before the lab.

Words we use today

	Word	What it means
	**Forward pass**	Computing the output and the loss from the inputs, layer by layer.
	**Backward pass**	Computing the gradient of the loss for every parameter, from the output back to the input.
	**Backpropagation**	The backward pass that applies the chain rule from the loss back to every parameter, reusing the values stored in the forward pass.
	**Delta**	The gradient of the loss with respect to the input of a unit, written with the Greek letter delta.
	**Sigmoid**	The function 1 / (1 + e^-z). It squeezes any number into the range 0 to 1.

A tiny network: 2 inputs, 2 hidden units with a sigmoid, and 1 output



The numbers on the lines are the weights.

Nine numbers can change: 6 weights and 3 biases.

A hidden unit adds its weighted inputs and bias, then applies the sigmoid.

The output unit adds its weighted inputs and its bias and applies no sigmoid. The target is y = 1.0.

Every number we start with: the inputs, the target and the nine parameters

Name	Meaning	Value
x	The two inputs of the example	1.0 and 2.0
y	The target (the answer we want)	1.0
W(1)	Weights into the hidden units (layer 1): one row per hidden unit	(0.2, -0.3) and (0.4, 0.1)
b(1)	Biases of the two hidden units	0.1 and -0.2
W(2)	Weights into the output unit (layer 2)	0.5 and -0.4
b(2)	Bias of the output unit	0.2
eta	The learning rate	0.5

These are illustrative numbers, chosen small and simple. Results are rounded to 4 decimals on the slides; the computer keeps full precision.

The forward pass: weighted sum, sigmoid, weighted sum, then the loss

Hidden unit j

$$z_j^{(1)} = W_{j1}^{(1)} x_1 + W_{j2}^{(1)} x_2 + b_j^{(1)} \qquad a_j^{(1)} = \sigma(z_j^{(1)}) = \frac{1}{1 + e^{-z_j^{(1)}}}$$

Output

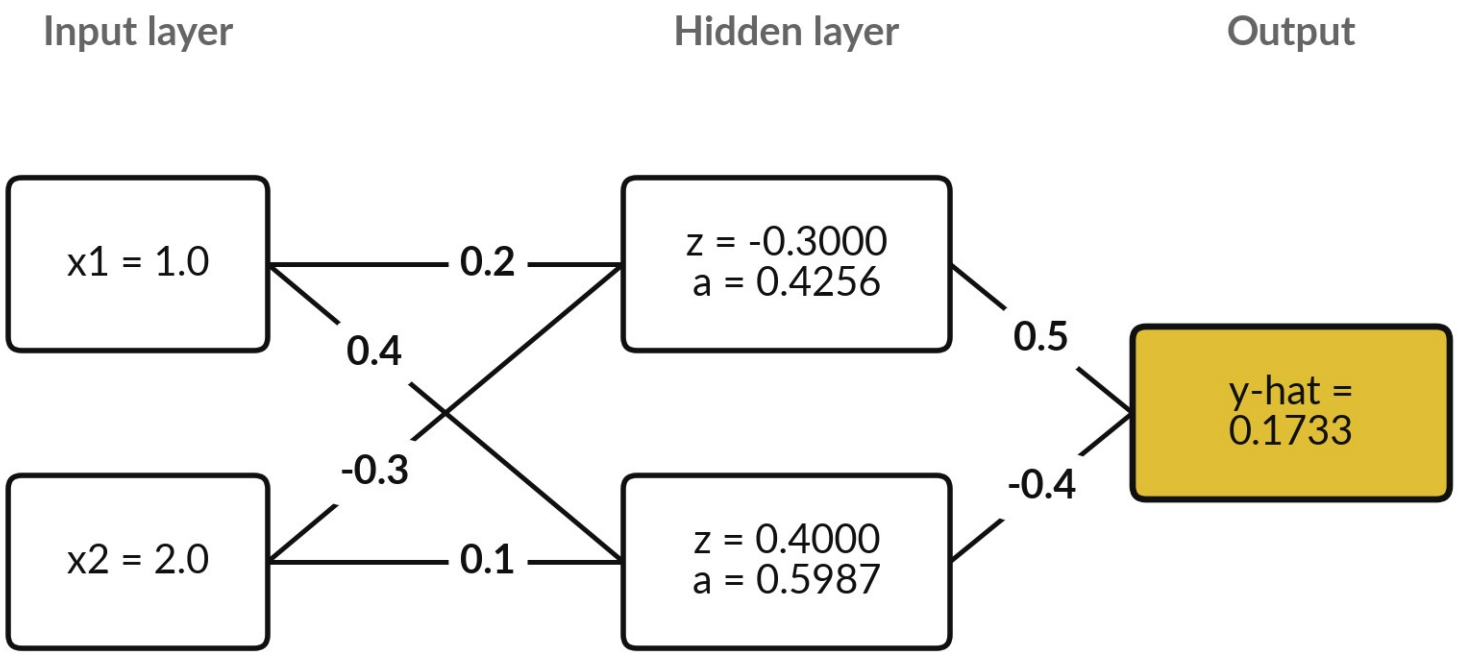
$$\hat{y} = W_1^{(2)} a_1^{(1)} + W_2^{(2)} a_2^{(1)} + b^{(2)}$$

Loss

$$L = \tfrac{1}{2}(\hat{y} - y)^2$$

Symbol	What it means	In our data
z(1)_j, a(1)_j	The input of hidden unit j in layer 1 (weighted sum plus bias) and its output after the sigmoid. Without the subscript, z(1) and a(1) mean the list of both units.	j = 1, 2
W(1), b(1)	Weights and biases of layer 1, with W(1)_ji from input i to hidden unit j. The layer is in brackets, the unit is the subscript. In code: W1, b1.	the table of starting numbers
y-hat, L	The prediction, and the loss (the half makes the derivative simple).	one number each

The forward pass with our numbers: the loss is 0.3417



$$L = 0.5 \times (0.1733 - 1.0)^2 = 0.3417$$

The prediction is 0.1733, far from the target 1.0.

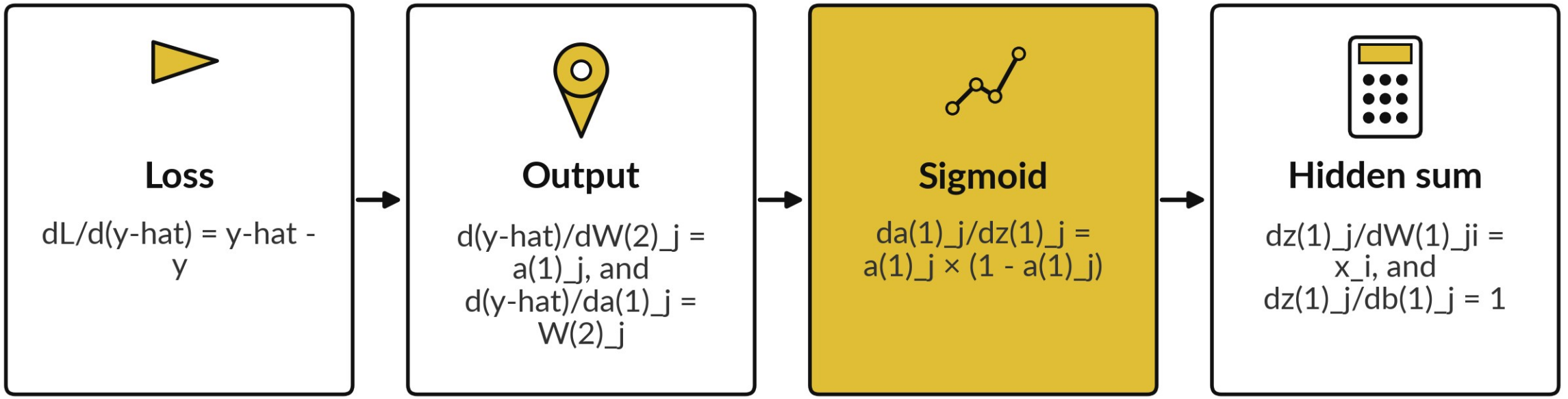
z is the weighted sum in a hidden unit; a is its sigmoid.

The loss is large, so the gradients will push the weights hard.

The forward pass, one calculation at a time

Quantity	Calculation	Value
z(1)_1	0.2 × 1.0 + (-0.3) × 2.0 + 0.1	-0.3000
a(1)_1 = sigmoid(z(1)_1)	1 / (1 + e^(0.3000))	0.4256
z(1)_2	0.4 × 1.0 + 0.1 × 2.0 + (-0.2)	0.4000
a(1)_2 = sigmoid(z(1)_2)	1 / (1 + e^(-0.4000))	0.5987
y-hat	0.5 × 0.4256 + (-0.4) × 0.5987 + 0.2	0.1733
L	0.5 × (0.1733 - 1.0)^2	0.3417

The backward pass multiplies local slopes along each path from the loss back to a weight



Chain rule from Session 9: the slope for a weight is the product of the slopes along the path.

Backward pass, part 1: the output layer

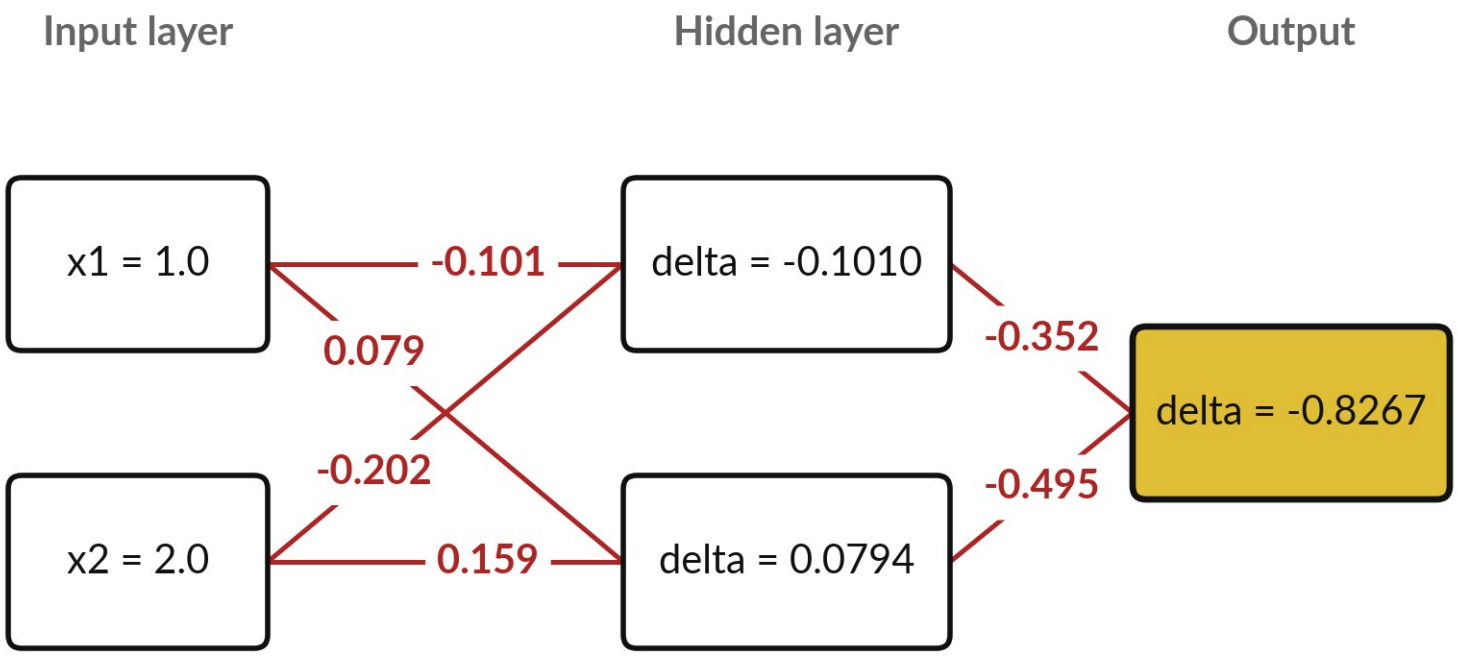
Quantity	Calculation	Value
delta(2) = dL/d(y-hat)	0.1733 - 1.0	-0.8267
dL/dW(2)_1 (first weight)	delta(2) × a(1)_1 = (-0.8267) × 0.4256	-0.3518
dL/dW(2)_2 (second weight)	delta(2) × a(1)_2 = (-0.8267) × 0.5987	-0.4949
dL/db(2)	delta(2) × 1	-0.8267

delta(2) is the gradient of the loss with respect to the output y-hat. Negative values mean: raise this parameter to lower the loss.

Backward pass, part 2: the hidden layer

Quantity	Calculation	Value
dL/da(1)_1	W(2)_1 × delta(2) = 0.5 × (-0.8267)	-0.4133
delta(1)_1	(-0.4133) × a(1)_1 × (1 - a(1)_1) = (-0.4133) × 0.2445	-0.1010
dL/da(1)_2	W(2)_2 × delta(2) = (-0.4) × (-0.8267)	0.3307
delta(1)_2	0.3307 × a(1)_2 × (1 - a(1)_2) = 0.3307 × 0.2403	0.0794
dL/dW(1) (row 1)	delta(1)_1 × inputs = (-0.1010) × (1.0, 2.0)	-0.1010, -0.2021
dL/dW(1) (row 2)	delta(1)_2 × inputs = 0.0794 × (1.0, 2.0)	0.0794, 0.1589
dL/db(1)	the two values of delta(1)	-0.1010, 0.0794

All nine gradients, drawn on the network



Red numbers: dL/d(weight). Boxes: delta. Bias gradients equal the deltas.

Each red number is the slope of the loss for that weight: the change in loss per unit change in the weight, for a tiny step.

Negative gradient: raise that weight to lower the loss.

The output layer has the largest gradients.

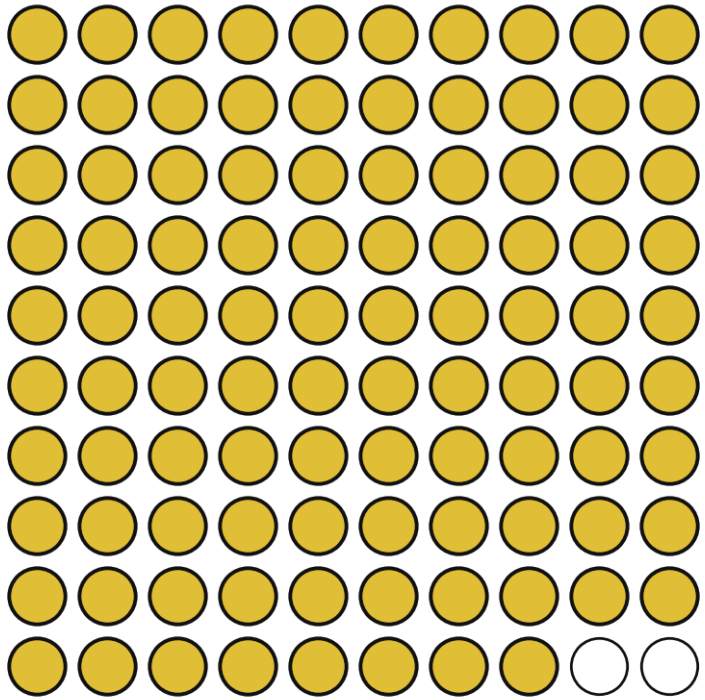
The update: every parameter moves by minus eta times its gradient, with eta = 0.5

Parameter	Old value	Gradient	New value = old - 0.5 × gradient
W(1) row 1, input 1	0.2000	-0.1010	0.2505
W(1) row 1, input 2	-0.3000	-0.2021	-0.1990
W(1) row 2, input 1	0.4000	0.0794	0.3603
W(1) row 2, input 2	0.1000	0.1589	0.0206
b(1) unit 1	0.1000	-0.1010	0.1505
b(1) unit 2	-0.2000	0.0794	-0.2397
W(2), hidden 1	0.5000	-0.3518	0.6759
W(2), hidden 2	-0.4000	-0.4949	-0.1525
b(2)	0.2000	-0.8267	0.6133

After one update the loss falls from 0.3417 to 0.0085

98%

of the loss is gone after a single step (rounded to the nearest percent). The new prediction is 0.8694, close to the target 1.0.



of the loss removed

Source: Computed from the illustrative network of this session. A second step brings the loss to 0.0002.

In matrix form the same computation uses W, b and delta

Output delta

$$\delta^{(2)} = \hat{y} - y \qquad \nabla W^{(2)} = \delta^{(2)} a^{(1)T} \qquad \nabla b^{(2)} = \delta^{(2)}$$

Hidden delta

$$\delta^{(1)} = (W^{(2)T}\delta^{(2)}) \odot a^{(1)} \odot (1 - a^{(1)})$$

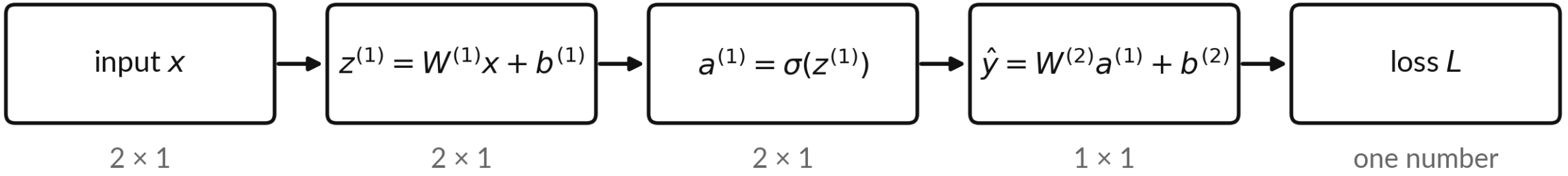
Hidden gradients

$$\nabla W^{(1)} = \delta^{(1)} x^{T} \qquad \nabla b^{(1)} = \delta^{(1)}$$

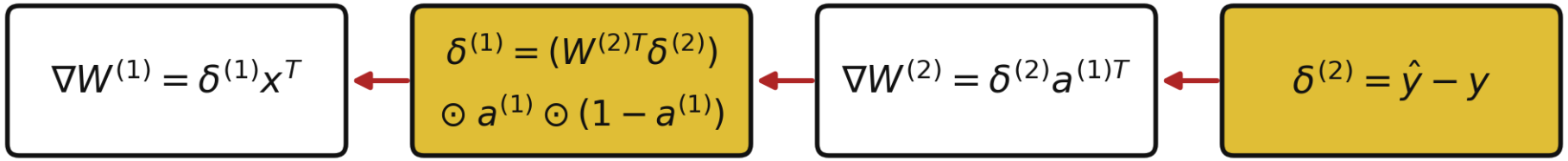
Symbol	What it means	In our data
delta(2), delta(1)	The deltas of the output layer and of the hidden layer.	-0.8267; (-0.1010, 0.0794)
T	Transpose: rows become columns.	W(2) is 1 × 2, so W(2)^T is 2 × 1
⊙	The Hadamard (element-wise) product: multiply two vectors entry by entry.	a(1) ⊙ (1 - a(1)) is the sigmoid slope
nabla W	The matrix of gradients, the same shape as W.	nabla W(1) is 2 × 2

The forward pass stores every value, and the backward pass reuses them

Forward pass: left to right, and store every value



Backward pass: right to left, reusing the stored values



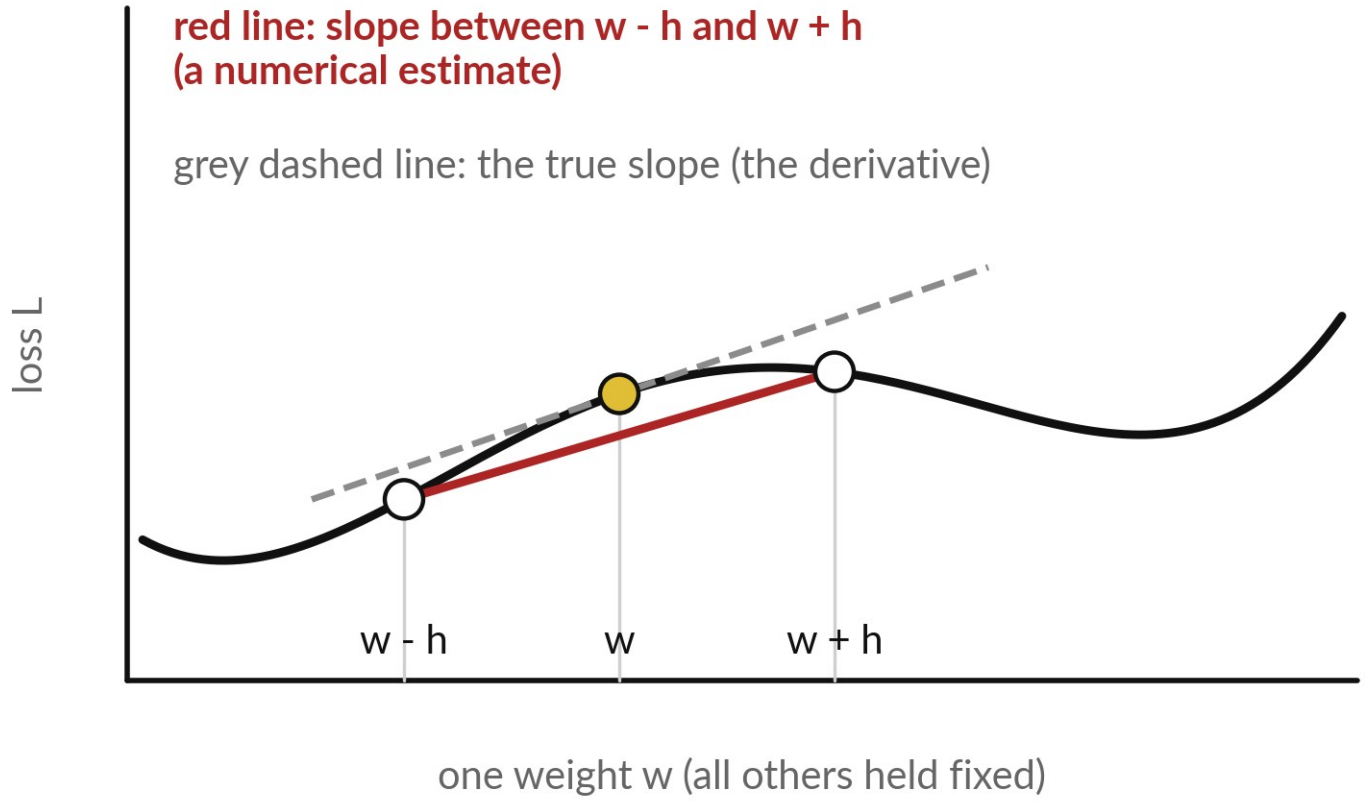
Top: forward pass with the shape of each value. Bottom: backward pass. Gold boxes are the deltas.

Shapes in the matrix form: check them before you code

Object	Shape	Why
x, z(1), a(1)	2 × 1	Two inputs, two hidden units
W(1)	2 × 2	One row per hidden unit, one column per input
W(2)	1 × 2	One output, two hidden units
delta(2) and y-hat	1 × 1	One output
delta(1)	2 × 1	One per hidden unit
nabla W(2) = delta(2) a(1)^T	(1 × 1)(1 × 2) = 1 × 2	Same shape as W(2)
nabla W(1) = delta(1) x^T	(2 × 1)(1 × 2) = 2 × 2	Same shape as W(1)

Rule: the gradient of a parameter always has the same shape as the parameter. For a batch of n examples the gradients are averaged over the n rows.

Check a gradient with finite differences: nudge one weight and watch the loss



A finite difference estimates a slope from two nearby values of the loss.

Nudge one weight by +h and -h. Keep all others fixed.

The red slope should match backpropagation. The check is slow, so it is only a test.

The central finite difference, checked on one weight of our network

Central difference

$$\frac{\partial L}{\partial w} \approx \frac{L(w+h) - L(w-h)}{2h}$$

Symbol	What it means	In our data
w	The first weight of the first layer, W(1) row 1, input 1.	0.2
h	A tiny nudge.	0.00001
Finite difference	The estimate from the formula.	-0.1010463936
Backpropagation	The gradient from the backward pass.	-0.1010463936
Difference	The two estimates agree to about 10 decimals.	3.5e-12

Rule of thumb: a relative difference below about 1e-6 is a good sign, but it proves nothing by itself.

Four mistakes in backpropagation, what you see, and the fix

Mistake	What you see	Fix
Update the weights before all gradients are computed	The gradients of earlier layers are wrong	Compute all gradients first, then update everything
Forget the sigmoid slope a × (1 - a)	Hidden gradients are too large and fail the finite-difference check	Multiply by a × (1 - a) entry by entry
A missing transpose	A shape error in numpy, or a result of the wrong shape	The gradient has the shape of its parameter
A sign error in delta(2)	The loss goes up after the update	delta(2) = y-hat - y, and the update subtracts

Read the training step before you write it: forward, backward, update

```
z1 = W1 @ x + b1
a1 = 1 / (1 + np.exp(-z1))
yhat = W2 @ a1 + b2
d2 = yhat - y
d1 = (W2.T @ d2) * a1 * (1 - a1)
W2 = W2 - eta * np.outer(d2, a1)
W1 = W1 - eta * np.outer(d1, x)
```

In plain words

In code, z1, a1, W1 and b1 stand for z(1), a(1), W(1) and b(1); d2 and d1 stand for delta(2) and delta(1).

The first three lines are the forward pass; d2 and d1 are the backward pass.

np.outer(a, b) is a times b-transposed.

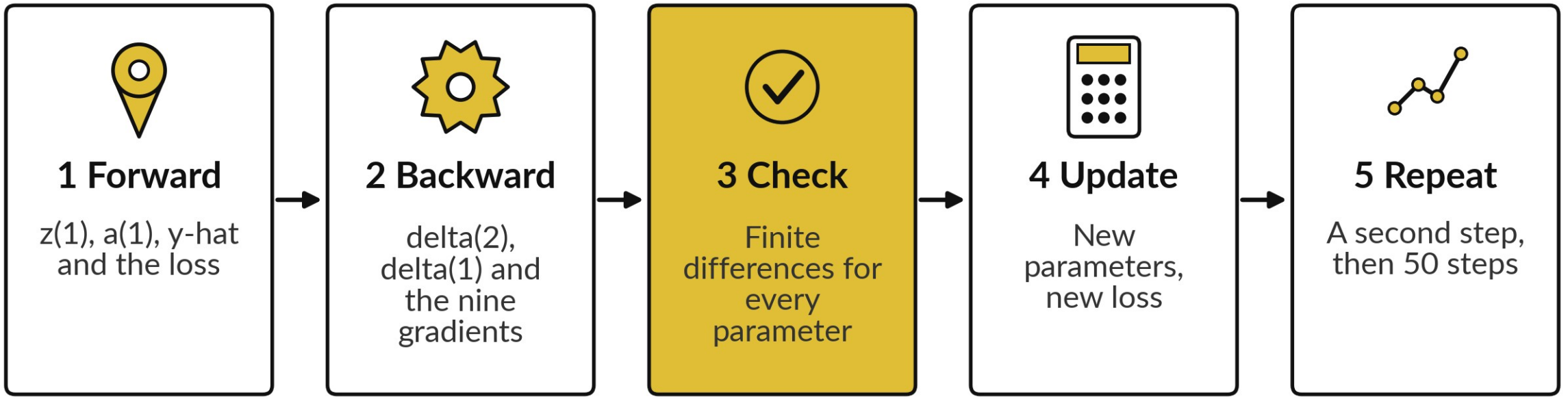
The last lines are the update. The biases follow the same pattern.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

The lab in five stages, in Python and in the spreadsheet



Predict first, run, read, then change one thing. The check cells tell you when a number is right.

Check your numbers against these before you move on

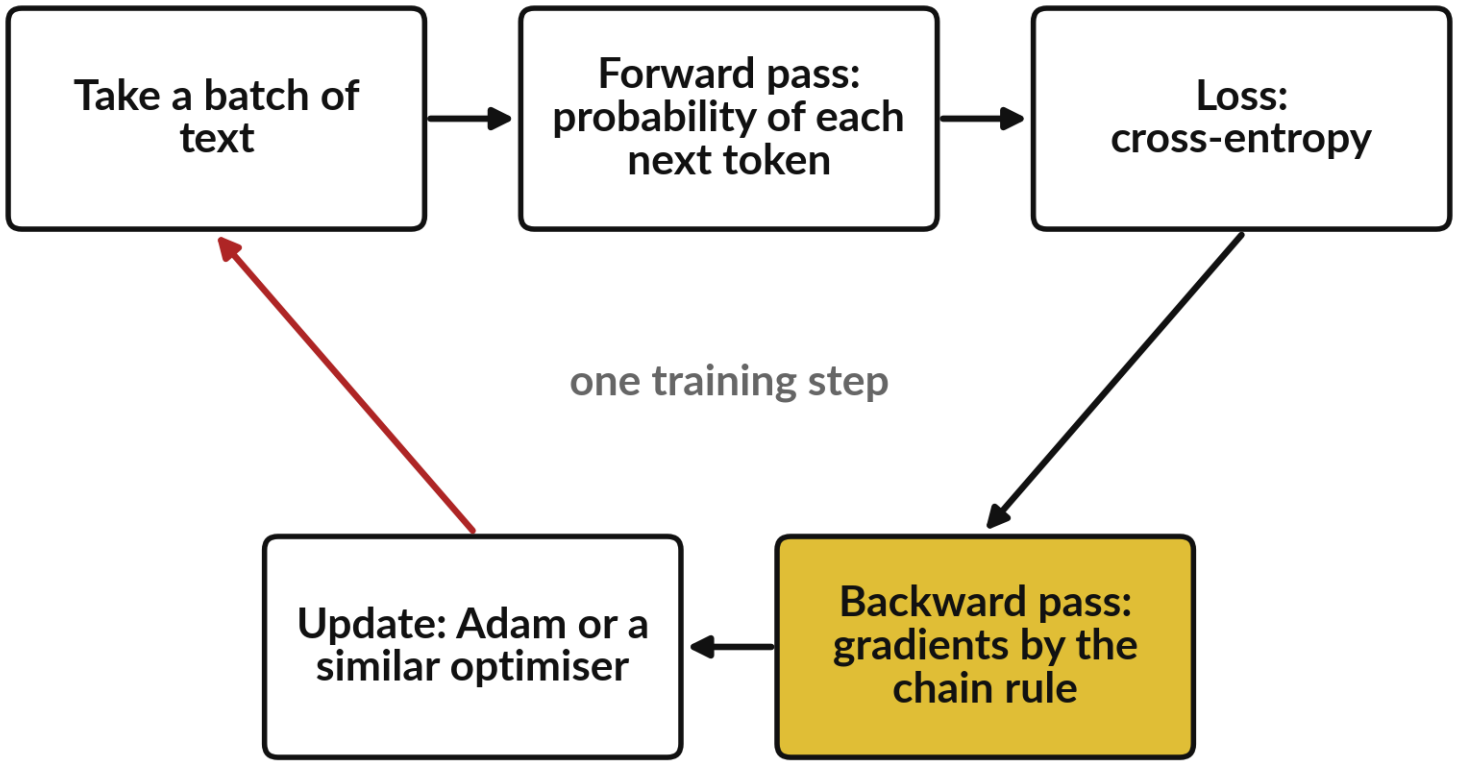
What you compute	You should see
z(1) (both hidden units)	-0.3000 and 0.4000
a(1) (both hidden units)	0.4256 and 0.5987
Prediction y-hat, loss L	0.1733, 0.3417
delta(2), then delta(1) (two values)	-0.8267; -0.1010 and 0.0794
Gradient of W(1) row 1 (two values)	-0.1010 and -0.2021
New W(1) row 1 after the update	0.2505 and -0.1990
Loss after one update, after two	0.0085 and 0.0002

If your number differs, find the step that went wrong. Do not change a number to match.

Errors you will meet in the lab, what they mean and what to do

	What you see	What it means	What to do
	**ValueError: shapes not aligned**	A matrix product has the wrong shape	Print the shapes. The gradient has the shape of its parameter
	**Finite-difference check shows a large difference**	One gradient formula is wrong	Compare parameter by parameter to find which one
	**The loss goes up after the update**	A sign error, or eta is too large	delta(2) = y-hat - y; update is minus eta × gradient
	**NameError**	A cell that defines a name was not run	Run all cells from the top

Where this appears in AI: every training step is a forward pass, a loss and a backward pass

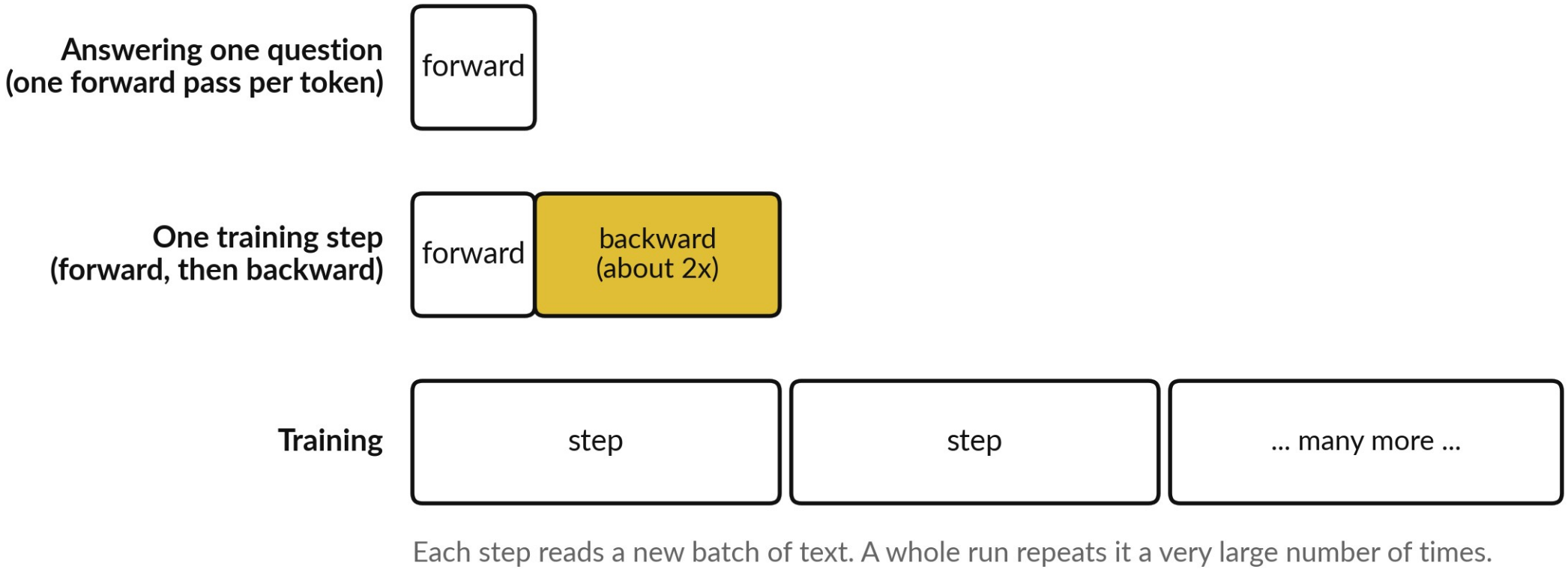


The loop you did by hand is the loop used for large models.

Published work describes these steps for models of this kind.

Only the sizes differ. Product designs are not published.

Where this appears in AI: why training needs far more computing than answering



Illustrative picture. Published analyses count the backward pass as about twice the forward pass.

Check an AI assistant with your own numbers

10 minutes Hands-on, with no private data

Ask any AI assistant to run the forward pass of our network. Then compare with your table.

Give it x = (1.0, 2.0), the table of parameters, and target 1. Ask for z, a, y-hat and L.

Expected y-hat: 0.1733.

If it differs, find the first number that differs.

Never paste confidential data.

You keep

A note: did the assistant match your table?

Exit check: three questions with short answers

Question	Short answer
What is delta(2) in this network, in words and as a formula?	The slope of the loss with respect to the prediction: y-hat - y. For us it was -0.8267.
Why does the hidden delta contain a × (1 - a)?	It is the slope of the sigmoid. The chain rule multiplies it into the slope that arrives from the output.
Give two reasons training costs more computing than answering.	Each step needs a backward pass as well as a forward pass, and training repeats the step a very large number of times.

Save your lab and keep the nine gradients

What you keep from today

Keep your saved notebook or workbook, with the nine gradients of the tiny network and the check that finite differences agree with them. Write down the loss before and after one update.

Our question today

What happens in every training step of a language model, and why does training cost far more computing than answering a question?

Next session

Session 12: optimisation in practice: regularisation, constraints, overflow and the stable softmax.

Take away

Loss, before
0.3417

Loss, after one update
0.0085

Check
Finite differences agree

Next session
Optimisation in practice